Real-Time MPU6050 Inertial Sensing and 3D Motion Visualization Using Arduino Mega and Python

Zhang YUE
Mechanical Engineering and Robotics
Guangdong Technion–Israel Institute of Technology (GTIIT)
yue24382@gtiit.edu.cn

May 21, 2026

Contents

1. Introduction **2**
2. Hardware Setup **3**
3. Embedded Sensor Acquisition **4**
3.1 MPU6050 Configuration 4
3.2 Six-Axis Reading and Serial Transmission 5
4. Host-Side Reception and Unit Conversion **5**
5. Initial Calibration **6**
6. Attitude Estimation on SO(3) **6**
6.1 Skew-Symmetric Matrix and Exponential Map 6
6.2 Gyroscope Propagation 7
7. Acceleration Integration and Position Estimate **8**
8. Gravity Correction and Drift Control **8**
9. Real-Time Visualization **9**
10.Discussion and Limitations **10**
Appendix **11**

Abstract

This report presents a real-time inertial sensing and three-dimensional visualization framework based on an MPU6050 six-axis inertial measurement unit, an Arduino Mega microcontroller, and a Python OpenGL visualization program. The system configures the MPU6050 for digital low-pass filtering, 500 Hz sampling, a gyroscope range of ± 250 deg/s, and an accelerometer range of $\pm 2g$. The Arduino program reads the six-axis raw sensor output and transmits it to the host computer through serial communication. The Python program receives the serial stream, converts raw measurements into physical units, performs initial calibration, integrates angular velocity on $SO(3)$, estimates qualitative translational motion from acceleration, and renders a real-time 3D model. Experimental results show that the virtual model follows the physical orientation of the MPU6050 in real time. Position drift is also observed, which is expected because acceleration is double-integrated without an external position reference. The project demonstrates a complete sensor-integrated pipeline from hardware sensing to real-time visual feedback.

1. Introduction

Inertial measurement units are widely used in robotics, drones, smartphones, automotive systems, virtual reality devices, and attitude-control systems. The MPU6050 is a low-cost six-axis IMU that integrates a three-axis accelerometer and a three-axis gyroscope. The accelerometer measures specific force along three orthogonal axes, while the gyroscope measures angular velocity about the same axes. By processing these two types of measurements, the orientation and qualitative motion of the sensor can be estimated.

The objective of this project is to implement a complete real-time sensing and visualization system. The system begins with embedded sensor acquisition on the Arduino Mega, continues through serial communication, and ends with host-side motion estimation and OpenGL rendering. The implemented pipeline can be summarized as

$$\text{MPU6050} \rightarrow \text{Arduino} \rightarrow \text{Serial} \rightarrow \text{Python} \rightarrow \text{Estimation} \rightarrow \text{3D Visualization}. \quad (1)$$

The main goal is real-time orientation visualization. Position is also estimated by integrating acceleration, but it is treated as a qualitative visual cue rather than an accurate navigation result, because low-cost IMU position estimation suffers from accumulated drift.

This project requires MPU6050 configuration, six-axis IMU reading, serial data transmission, host-side reception, integration of rotational velocity and acceleration, and visualization. Table 1 maps these requirements to the implemented system.

Table 1: Requirements

Requirement	Implementation	Explanation
Configure MPU6050	Arduino writes to MPU6050 registers	DLPF, 500 Hz sample rate, ± 250 deg/s gyroscope range, and $\pm 2g$ accelerometer range are set.
Read six-axis data	Arduino requests 14 bytes from `REG_ACCEL_XOUT_H`	Three accelerometer values and three gyroscope values are extracted while temperature bytes are skipped.
Send data	Arduino prints CSV samples	Each line contains `ax,ay,az,gx,gy,gz`.
Receive data	Python serial reader parses CSV lines	Valid samples are timestamped and placed in a queue.
Estimate motion	Python estimator integrates angular velocity and acceleration	Attitude and qualitative position are estimated.
Visualize result	PyQt6/OpenGL viewer renders a model	The virtual model follows the physical IMU motion.

2. Hardware Setup

The hardware system consists of an Arduino Mega 2560, an MPU6050 module, jumper wires, and a laptop computer. The MPU6050 communicates with the Arduino Mega using the I2C protocol. For the Arduino Mega, the I2C pins are SDA = 20 and SCL = 21. The wiring is

$$\begin{aligned} &\text{VCC} \rightarrow 5\text{ V}, \qquad \text{GND} \rightarrow \text{GND}, \\ &\text{SDA} \rightarrow \text{Pin } 20, \quad \text{SCL} \rightarrow \text{Pin } 21, \\ &\text{AD0} \rightarrow \text{GND}. \end{aligned} \tag{2}$$

Connecting AD0 to ground sets the MPU6050 I2C address to $0x68$. The physical hardware setup is shown in Fig. 1.

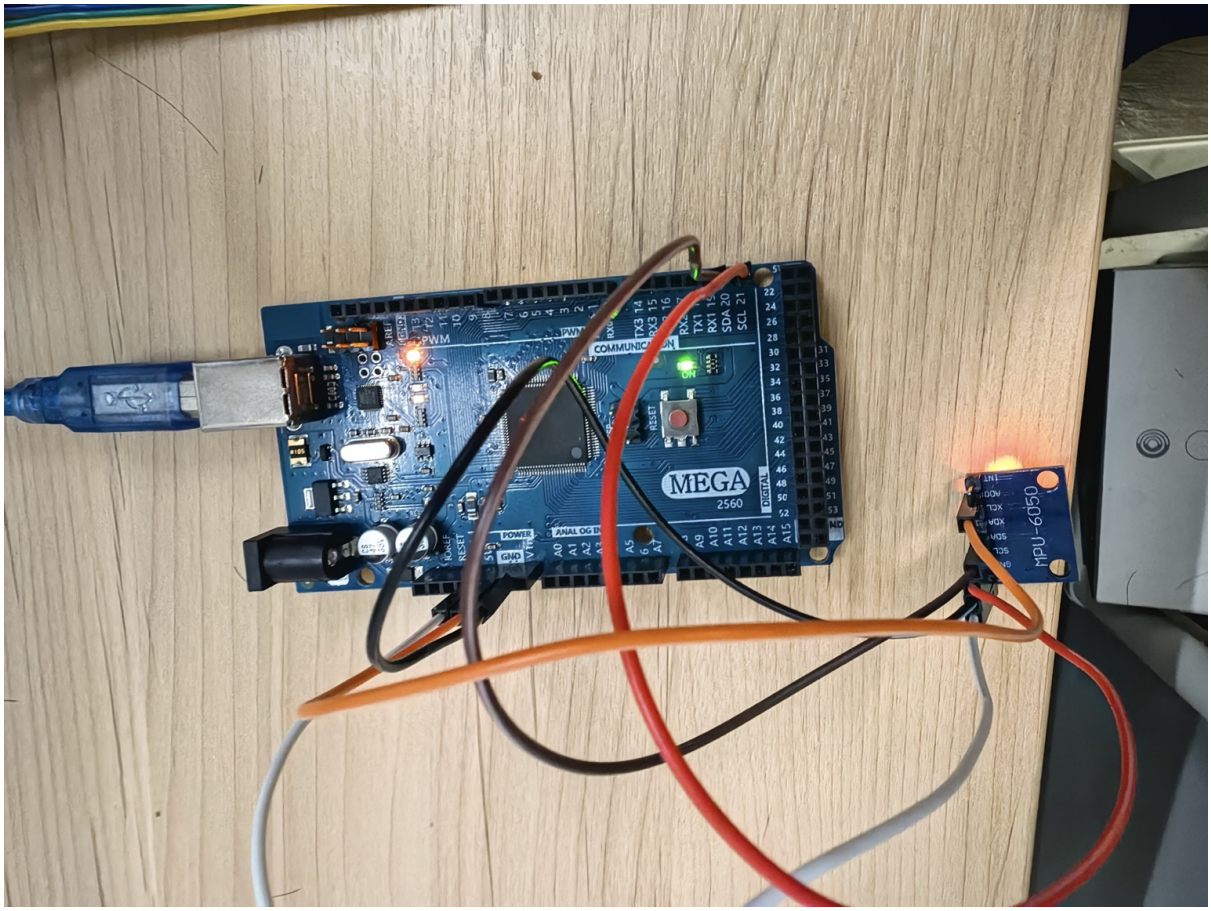

Figure 1: Hardware setup of the MPU6050 module connected to the Arduino Mega through I2C.

3. Embedded Sensor Acquisition

3.1 MPU6050 Configuration

The Arduino program wakes up the MPU6050 and configures the digital low-pass filter, sample-rate divider, gyroscope range, and accelerometer range. The required register settings are

$$\begin{aligned} \text{REG\_CONFIG} &= 0x03, \\ \text{REG\_SMPLRT\_DIV} &= 1, \\ \text{REG\_GYRO\_CONFIG} &= 0x00, \\ \text{REG\_ACCEL\_CONFIG} &= 0x00. \end{aligned} \tag{3}$$

With the DLPF enabled, the internal output rate is treated as 1000 Hz. The sample rate is therefore

$$f_s = \frac{1000}{1 + \text{SMPLRT\_DIV}} = \frac{1000}{2} = 500 \text{ Hz}. \tag{4}$$

The corresponding sampling period is

$$T_s = \frac{1}{f_s} = 0.002 \text{ s} = 2000 \ \mu\text{s}. \tag{5}$$

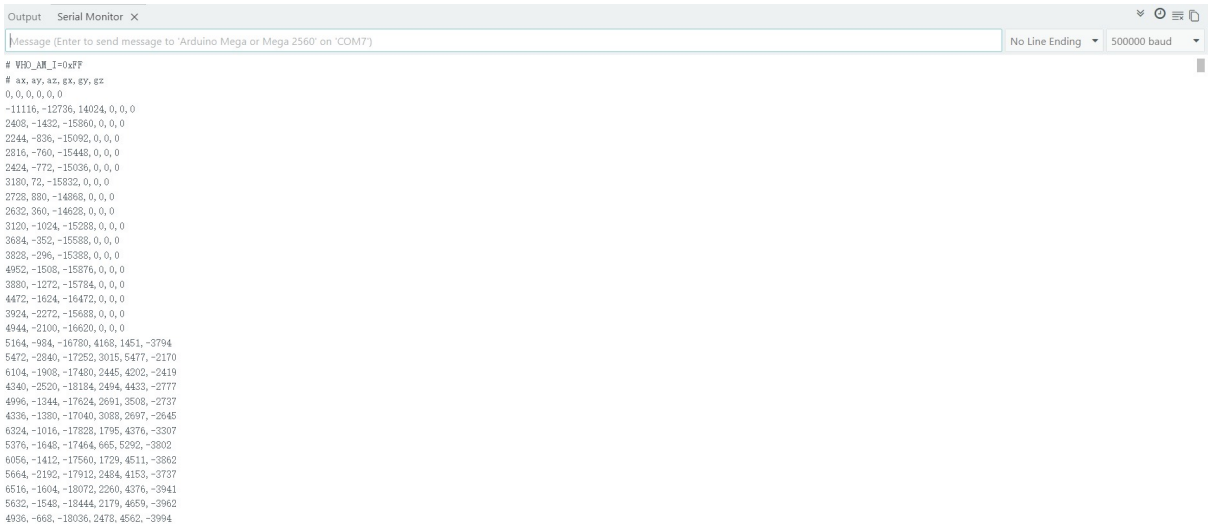

Figure 2: Arduino serial monitor output showing real-time six-axis MPU6050 measurements.

The Arduino code enforces this timing using `SAMPLE_PERIOD_US = 2000`.

3.2 Six-Axis Reading and Serial Transmission

The Arduino requests 14 consecutive bytes from `REG_ACCEL_XOUT_H`. These bytes contain three accelerometer channels, two temperature bytes, and three gyroscope channels. The temperature bytes are skipped because they are not required for the visualization task. At sample index k, the raw measurement vector is

$$\mathbf{y}_{raw,k} = \begin{bmatrix} a^r_{x,k} & a^r_{y,k} & a^r_{z,k} & g^r_{x,k} & g^r_{y,k} & g^r_{z,k} \end{bmatrix}^T. \tag{6}$$

Each sample is transmitted as a comma-separated line. The serial monitor result in Fig. 2 confirms that the system continuously streams six-axis IMU data.

4. Host-Side Reception and Unit Conversion

The Python program opens the serial port, reads each line, ignores invalid or comment lines, and parses valid lines into six integers. A thread-safe queue separates high-rate serial reception from the OpenGL rendering loop.

The MPU6050 is configured for $\pm 2g$ acceleration and ± 250 deg/s angular velocity. The standard sensitivities are

$$S_a = 16384 \text{ LSB}/g, \qquad S_g = 131 \text{ LSB}/(\text{deg/s}). \tag{7}$$

The accelerometer data are converted into SI units by

$$\mathbf{a}_{m,k} = \frac{9.80665}{16384} \begin{bmatrix} a^r_{x,k} \\ a^r_{y,k} \\ a^r_{z,k} \end{bmatrix}. \tag{8}$$

The gyroscope data are converted into radians per second by

$$\boldsymbol{\omega}_{m,k} = \frac{\pi}{180 \cdot 131} \begin{bmatrix} g^r_{x,k} \\ g^r_{y,k} \\ g^r_{z,k} \end{bmatrix}. \tag{9}$$

The world gravity vector is defined as

$$\mathbf{g} = \begin{bmatrix} 0 & 0 & -9.80665 \end{bmatrix}^T \ \mathrm{m/s^2}. \tag{10}$$

The attitude matrix $\mathbf{R}_k \in SO(3)$ maps a body-frame vector to the world frame:

$$\mathbf{v}^w = \mathbf{R}_k \mathbf{v}^b. \tag{11}$$

5. Initial Calibration

Before motion estimation begins, the MPU6050 is kept still and the first $N_c = 250$ samples are used for calibration. The mean accelerometer and gyroscope measurements are

$$\bar{\mathbf{a}}_m = \frac{1}{N_c} \sum_{k=1}^{N_c} \mathbf{a}_{m,k}, \tag{12}$$

$$\bar{\boldsymbol{\omega}}_m = \frac{1}{N_c} \sum_{k=1}^{N_c} \boldsymbol{\omega}_{m,k}. \tag{13}$$

The gyroscope bias is estimated as

$$\mathbf{b}_g = \bar{\boldsymbol{\omega}}_m. \tag{14}$$

The initial rotation matrix is obtained by aligning the measured stationary acceleration with the opposite of gravity:

$$\mathbf{R}_0 = \mathrm{RotBetween}(\bar{\mathbf{a}}_m, -\mathbf{g}). \tag{15}$$

The accelerometer bias is estimated as

$$\mathbf{b}_a = \bar{\mathbf{a}}_m - \mathbf{R}_0^T(-\mathbf{g}). \tag{16}$$

The initial velocity and position are then set to zero:

$$\mathbf{v}_0 = \mathbf{0}, \qquad \mathbf{p}_0 = \mathbf{0}. \tag{17}$$

A stationary visualization state is shown in Fig. 3.

6. Attitude Estimation on SO(3)

6.1 Skew-Symmetric Matrix and Exponential Map

For a rotation vector

$$\boldsymbol{\phi} = \begin{bmatrix} \phi_x & \phi_y & \phi_z \end{bmatrix}^T, \tag{18}$$

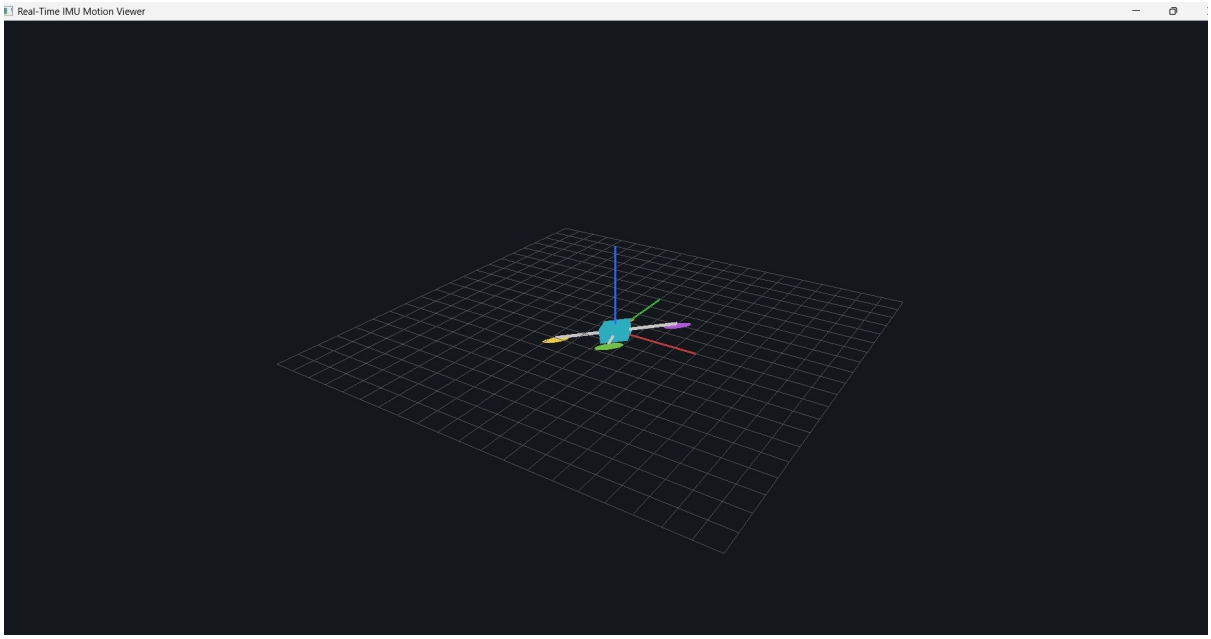

Figure 3: Visualization result when the MPU6050 is approximately stationary near the origin after calibration.

the corresponding skew-symmetric matrix is

$$[\boldsymbol{\phi}]_\times = \begin{bmatrix} 0 & -\phi_z & \phi_y \\ \phi_z & 0 & -\phi_x \\ -\phi_y & \phi_x & 0 \end{bmatrix}. \tag{19}$$

The exponential map is

$$\exp([\boldsymbol{\phi}]_\times) = \mathbf{I} + \frac{\sin\theta}{\theta}[\boldsymbol{\phi}]_\times + \frac{1-\cos\theta}{\theta^2}[\boldsymbol{\phi}]_\times^2, \tag{20}$$

where

$$\theta = \|\boldsymbol{\phi}\|. \tag{21}$$

For small θ, the approximation

$$\exp([\boldsymbol{\phi}]_\times) \approx \mathbf{I} + [\boldsymbol{\phi}]_\times \tag{22}$$

is used to avoid numerical singularity.

6.2 Gyroscope Propagation

The elapsed time is clipped to avoid unstable integration during serial timing irregularities:

$$\Delta t_k = \min\left(\max(t_k - t_{k-1}, 10^{-4}), 0.02\right). \tag{23}$$

After bias correction,

$$\boldsymbol{\omega}_k = \boldsymbol{\omega}_{m,k} - \mathbf{b}_g, \tag{24}$$

the attitude is propagated by

$$\mathbf{R}_{k+1} = \mathbf{R}_k \exp\left([\boldsymbol{\omega}_k \Delta t_k]_\times\right). \tag{25}$$

7. Acceleration Integration and Position Estimate

After accelerometer bias correction,

$$\mathbf{a}_k = \mathbf{a}_{m,k} - \mathbf{b}_a. \tag{26}$$

The body-frame specific force is transformed to the world frame and gravity is added:

$$\mathbf{a}_{w,k} = \mathbf{R}_{k+1}\mathbf{a}_k + \mathbf{g}. \tag{27}$$

The velocity and position are updated by discrete integration:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t_k + \frac{1}{2}\mathbf{a}_{f,k}\Delta t_k^2, \tag{28}$$

$$\mathbf{v}_{k+1} = \lambda(\mathbf{v}_k + \mathbf{a}_{f,k}\Delta t_k), \tag{29}$$

where λ is a damping coefficient and $\mathbf{a}_{f,k}$ is the acceleration used for integration. The displayed position is magnified for visualization:

$$\mathbf{p}_{display,k} = K_p \mathbf{p}_k. \tag{30}$$

8. Gravity Correction and Drift Control

Because pure gyroscope integration drifts, a gravity-based tilt correction is applied when the accelerometer magnitude is close to gravity and the angular velocity is small:

$$|\|\mathbf{a}_k\| - 9.80665| < 0.8, \tag{31}$$

$$\|\boldsymbol{\omega}_k\| < 25\frac{\pi}{180}. \tag{32}$$

The measured upward direction is

$$\hat{\mathbf{u}}_k = \mathbf{R}_k \frac{\mathbf{a}_k}{\|\mathbf{a}_k\|}, \tag{33}$$

and the target upward direction is

$$\mathbf{u}_g = \frac{-\mathbf{g}}{\|\mathbf{g}\|}. \tag{34}$$

The correction axis is

$$\mathbf{c}_k = \hat{\mathbf{u}}_k \times \mathbf{u}_g. \tag{35}$$

The correction gain is

$$\alpha_k = \min(K_g \Delta t_k, 0.08), \tag{36}$$

and the corrected attitude is

$$\mathbf{R}_k \leftarrow \exp([\alpha_k \mathbf{c}_k]_\times)\mathbf{R}_k. \tag{37}$$

This correction improves roll and pitch stability. Yaw remains weakly observable because the MPU6050 does not include a magnetometer or an external heading reference.

A deadband is applied to suppress small residual acceleration noise:

$$\mathbf{a}_{f,k} = \mathbf{0} \quad \text{if} \quad \|\mathbf{a}_{w,k}\| < a_{db}. \tag{38}$$

The code also includes stillness detection. When the IMU is classified as stationary, the estimator resets

$$\mathbf{p} \leftarrow \mathbf{0}, \qquad \mathbf{v} \leftarrow \mathbf{0}. \tag{39}$$

This improves visual stability, but it does not provide absolute position measurement.

9. Real-Time Visualization

The visualization program uses PyQt6 and OpenGL. The rendering scene contains a ground grid, coordinate axes, trajectory history, and a simplified 3D model. The attitude is updated from the estimated rotation matrix, and the position is updated from the integrated acceleration estimate.

Fig. 4 shows the visualization after the MPU6050 is moved. Fig. 5 shows a live demonstration in which both the physical MPU6050 module and the computer visualization are visible.

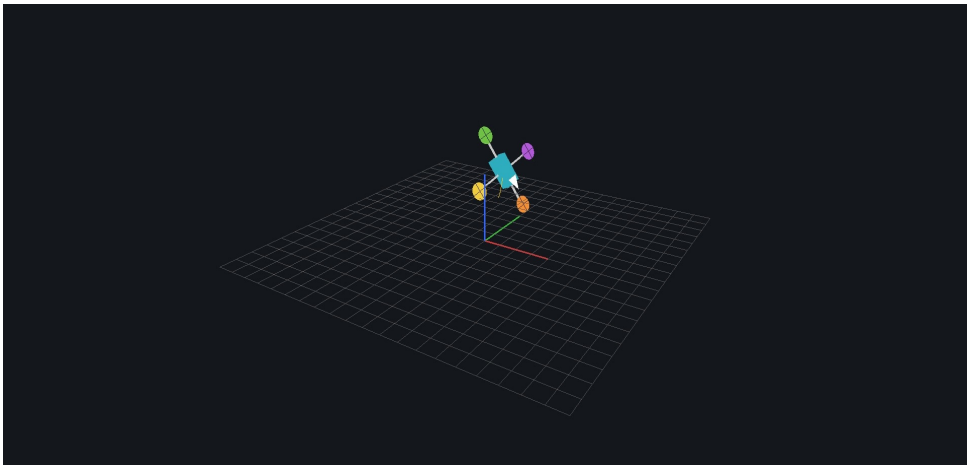

Figure 4: Real-time 3D visualization after moving the MPU6050. The model changes its position and orientation according to the estimated IMU state.

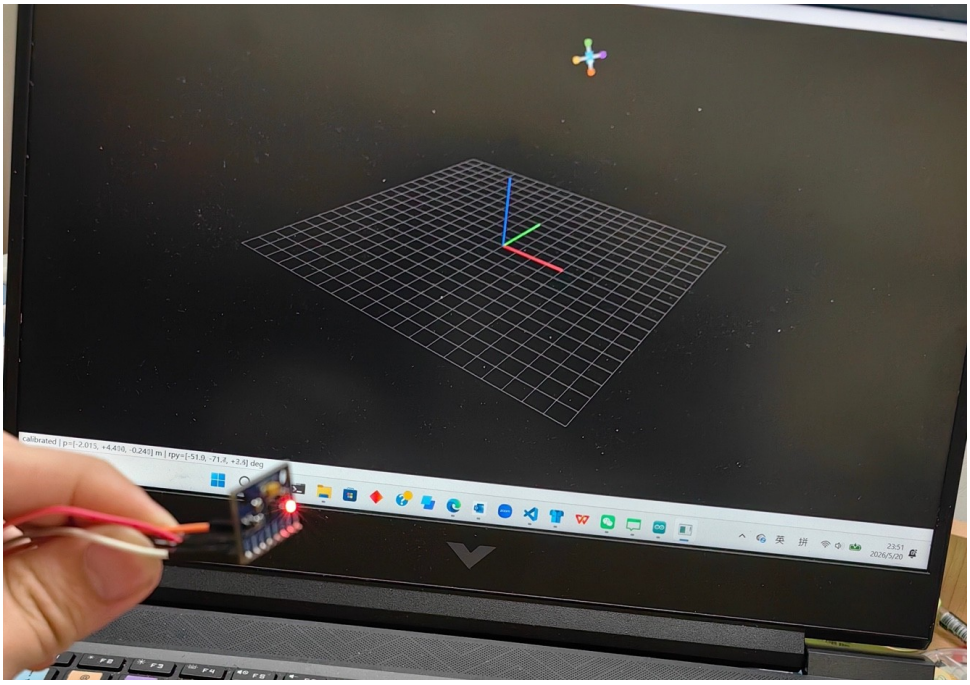

Figure 5: Live demonstration of synchronization between the physical MPU6050 module and the 3D visualization interface.

The experiment was conducted in three stages. First, the Arduino serial monitor was used to verify that the MPU6050 data stream was available. Second, the Python program was started while the MPU6050 was kept still for calibration. Third, the sensor was manually rotated and moved while the OpenGL model was observed.

The experimental observations are summarized as follows:

- The Arduino successfully transmitted six-axis IMU data.
- The Python program successfully received and parsed the serial data stream.
- The 3D model responded to the physical sensor orientation in real time.
- Roll, pitch, and yaw motions were visually reflected by the virtual model.
- Translational drift appeared during longer operation, which is expected for unaided IMU integration.

The main successful result is real-time orientation tracking. The virtual model follows the physical movement of the MPU6050, showing that the embedded acquisition, serial communication, numerical estimation, and visualization modules work together as an integrated system.

10. Discussion and Limitations

The main limitation is position drift. The MPU6050 does not directly measure absolute position. Position is obtained by double integration of acceleration:

$$\mathbf{v}(t) = \int \mathbf{a}(t)dt, \tag{40}$$

$$\mathbf{p}(t) = \int \mathbf{v}(t)dt = \int\int \mathbf{a}(t)dt^2. \tag{41}$$

If the acceleration has a small constant error ϵ, the position error grows approximately as

$$e_p(t) \approx \frac{1}{2}\epsilon t^2. \tag{42}$$

Therefore, even a small accelerometer bias, sensor noise, or gravity compensation error can make the model gradually move away from the origin. This explains why the virtual model may eventually fly out of the screen. This behavior is normal for pure inertial integration without GPS, optical tracking, UWB, visual odometry, or another external position reference.

The attitude estimate is more reliable than the position estimate because gravity provides a reference for roll and pitch during gentle motion. However, yaw remains weakly observable without a magnetometer or other heading reference. Therefore, this system should be described as a real-time visualization and sensing framework rather than a high-precision inertial navigation system.

References

[1] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification*, Rev. 3.4, Aug. 2013.

[2] Arduino, "Wire," Arduino Documentation. Available: `https://www.arduino.cc/en/Reference/Wire`. Accessed: May 2026.

[3] Khronos Group, "OpenGL," Khronos OpenGL Documentation. Available: `https://www.khronos.org/opengl/`. Accessed: May 2026.

[4] O. J. Woodman, *An Introduction to Inertial Navigation*, University of Cambridge Computer Laboratory, Technical Report UCAM-CL-TR-696, Aug. 2007.

Appendix

Complete Arduino Source Code

```
#include <Wire.h>

#define MPU_ADDR 0x68
#define REG_SMPLRT_DIV 0x19
#define REG_CONFIG 0x1A
#define REG_GYRO_CONFIG 0x1B
#define REG_ACCEL_CONFIG 0x1C
#define REG_ACCEL_XOUT_H 0x3B
#define REG_PWR_MGMT_1 0x6B
#define REG_WHO_AM_I 0x75

const unsigned long BAUD_RATE = 500000;
const unsigned long SAMPLE_PERIOD_US = 2000;

int16_t ax, ay, az, gx, gy, gz;
unsigned long next_sample_time = 0;

void writeMPU(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  Wire.write(value);
  Wire.endTransmission(true);
}

uint8_t readMPUReg(uint8_t reg) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, (uint8_t)1, (uint8_t)true);
  if (Wire.available()) return Wire.read();
  return 0xFF;
}

bool readMPU6Axis() {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(REG_ACCEL_XOUT_H);
  if (Wire.endTransmission(false) != 0) return false;

  uint8_t n = Wire.requestFrom(MPU_ADDR, (uint8_t)14, (uint8_t)true);
  if (n != 14) return false;

  ax = (int16_t)((Wire.read() << 8) | Wire.read());
```

```
  ay = (int16_t)((Wire.read() << 8) | Wire.read());
  az = (int16_t)((Wire.read() << 8) | Wire.read());

  Wire.read();
  Wire.read();

  gx = (int16_t)((Wire.read() << 8) | Wire.read());
  gy = (int16_t)((Wire.read() << 8) | Wire.read());
  gz = (int16_t)((Wire.read() << 8) | Wire.read());

  return true;
}

void configureMPU6050() {
  writeMPU(REG_PWR_MGMT_1, 0x00);
  delay(100);
  writeMPU(REG_CONFIG, 0x03);
  writeMPU(REG_SMPLRT_DIV, 0x01);
  writeMPU(REG_GYRO_CONFIG, 0x00);
  writeMPU(REG_ACCEL_CONFIG, 0x00);
}

void setup() {
  Serial.begin(BAUD_RATE);
  Wire.begin();
  Wire.setClock(400000);
  delay(500);

  configureMPU6050();

  Serial.print("# WHO_AM_I=0x");
  Serial.println(readMPUReg(REG_WHO_AM_I), HEX);
  Serial.println("# ax,ay,az,gx,gy,gz");

  next_sample_time = micros();
}

void loop() {
  unsigned long now = micros();

  if ((long)(now - next_sample_time) >= 0) {
    next_sample_time += SAMPLE_PERIOD_US;

    if (readMPU6Axis()) {
      Serial.print(ax); Serial.print(',');
      Serial.print(ay); Serial.print(',');
      Serial.print(az); Serial.print(',');
      Serial.print(gx); Serial.print(',');
      Serial.print(gy); Serial.print(',');
      Serial.println(gz);
    }
  }
}
```

Listing 1: Complete Arduino source code: MPU6050.ino.

Complete Python Visualization Source Code

```
from __future__ import annotations

import argparse
import csv
import math
import queue
import sys
import threading
import time
from collections import deque
from dataclasses import dataclass
from typing import Optional, Tuple

import numpy as np
import serial
from PyQt6 import QtCore, QtWidgets
from PyQt6.QtOpenGLWidgets import QOpenGLWidget
from OpenGL.GL import *
from OpenGL.GLU import *



G = np.array([0.0, 0.0, -9.80665], dtype=float)

ACC_SCALE = 9.80665 / 16384.0
GYRO_SCALE = math.pi / 180.0 / 131.0

CALIBRATION_SAMPLES = 250
MIN_DT = 1e-4
MAX_DT = 0.02
GRAVITY_CORRECTION_GAIN = 2.0
VELOCITY_DAMPING = 0.980
ACC_DEADBAND = 0.22
STILL_WINDOW_SEC = 0.35
RESET_COOLDOWN_SEC = 0.40
POSITION_DISPLAY_GAIN = 4.0


def normalize(v: np.ndarray, eps: float = 1e-12) -> np.ndarray:
    n = float(np.linalg.norm(v))
    if n < eps:
        return np.zeros_like(v)
    return v / n


def skew(v: np.ndarray) -> np.ndarray:
    x, y, z = v
    return np.array(
        [[0.0, -z, y],
         [z, 0.0, -x],
         [-y, x, 0.0]],
        dtype=float
    )


def exp_so3(phi: np.ndarray) -> np.ndarray:
    theta = float(np.linalg.norm(phi))
    K = skew(phi)

    if theta < 1e-8:
        return np.eye(3) + K

    return (
        np.eye(3)
        + math.sin(theta) / theta * K
        + (1.0 - math.cos(theta)) / (theta * theta) * (K @ K)
    )


def rot_between(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    a = normalize(a)
```

```
    b = normalize(b)

    c = float(np.clip(np.dot(a, b), -1.0, 1.0))

    if c > 1.0 - 1e-8:
        return np.eye(3)

    if c < -1.0 + 1e-8:
        axis = np.cross(a, np.array([1.0, 0.0, 0.0]))
        if np.linalg.norm(axis) < 1e-6:
            axis = np.cross(a, np.array([0.0, 1.0, 0.0]))
        return exp_so3(normalize(axis) * math.pi)

    axis = normalize(np.cross(a, b))
    angle = math.acos(c)
    return exp_so3(axis * angle)


def rpy_to_rot(roll: float, pitch: float, yaw: float) -> np.ndarray:
    cr, sr = math.cos(roll), math.sin(roll)
    cp, sp = math.cos(pitch), math.sin(pitch)
    cy, sy = math.cos(yaw), math.sin(yaw)

    Rx = np.array(
        [[1, 0, 0],
         [0, cr, -sr],
         [0, sr, cr]],
        dtype=float
    )

    Ry = np.array(
        [[cp, 0, sp],
         [0, 1, 0],
         [-sp, 0, cp]],
        dtype=float
    )

    Rz = np.array(
        [[cy, -sy, 0],
         [sy, cy, 0],
         [0, 0, 1]],
        dtype=float
    )

    return Rz @ Ry @ Rx


def rot_to_rpy(R: np.ndarray) -> Tuple[float, float, float]:
    pitch = math.asin(float(np.clip(-R[2, 0], -1.0, 1.0)))
    roll = math.atan2(R[2, 1], R[2, 2])
    yaw = math.atan2(R[1, 0], R[0, 0])
    return roll, pitch, yaw


@dataclass
class ImuSample:
    ax: int
    ay: int
    az: int
    gx: int
    gy: int
    gz: int
    t: float


class SerialWorker(threading.Thread):
    def __init__(self, port: str, baud: int, out_q: queue.Queue[ImuSample]):
        super().__init__(daemon=True)

        if serial is None:
            raise RuntimeError("pyserial is not installed. Run: pip install pyserial")

        self.port = port
```

```
145         self.baud = baud
146         self.out_q = out_q
147         self.running = True
148         self.ser: Optional[serial.Serial] = None
149 
150     def run(self) -> None:
151         self.ser = serial.Serial(self.port, self.baud, timeout=0.1)
152         time.sleep(2.0)
153         self.ser.reset_input_buffer()
154 
155         while self.running:
156             try:
157                 line = self.ser.readline().decode("ascii", errors="ignore").strip()
158 
159                 if not line or line.startswith("#"):
160                     continue
161 
162                 parts = line.split(",")
163 
164                 if len(parts) != 6:
165                     continue
166 
167                 values = [int(p.strip()) for p in parts]
168                 sample = ImuSample(*values, time.time())
169 
170                 if self.out_q.full():
171                     try:
172                         self.out_q.get_nowait()
173                     except queue.Empty:
174                         pass
175 
176                 self.out_q.put_nowait(sample)
177 
178             except Exception:
179                 continue
180 
181     def stop(self) -> None:
182         self.running = False
183 
184         if self.ser is not None:
185             self.ser.close()
186 
187 
188 class DemoSource(QtCore.QObject):
189     new_sample = QtCore.pyqtSignal(object)
190 
191     def __init__(self):
192         super().__init__()
193 
194         self.t0 = time.time()
195         self.timer = QtCore.QTimer()
196         self.timer.timeout.connect(self.emit_sample)
197         self.timer.start(2)
198 
199     def emit_sample(self) -> None:
200         t = time.time() - self.t0
201 
202         roll = 0.55 * math.sin(2.0 * t)
203         pitch = 0.40 * math.sin(2.7 * t + 0.3)
204         yaw = 0.75 * math.sin(1.5 * t)
205 
206         R = rpy_to_rot(roll, pitch, yaw)
207 
208         a_world = np.array(
209             [
210                 0.45 * math.sin(2.1 * t),
211                 0.30 * math.sin(3.0 * t),
212                 0.20 * math.sin(1.7 * t),
213             ],
214             dtype=float
215         )
216 
217         f_body = R.T @ (a_world - G)
```

```
218
219         gyro_body = np.array(
220             [
221                 0.55 * 2.0 * math.cos(2.0 * t),
222                 0.40 * 2.7 * math.cos(2.7 * t + 0.3),
223                 0.75 * 1.5 * math.cos(1.5 * t),
224             ],
225             dtype=float
226         )
227
228         acc_raw = np.round(f_body / ACC_SCALE).astype(int)
229         gyro_raw = np.round(gyro_body / GYRO_SCALE).astype(int)
230
231         self.new_sample.emit(
232             ImuSample(
233                 acc_raw[0],
234                 acc_raw[1],
235                 acc_raw[2],
236                 gyro_raw[0],
237                 gyro_raw[1],
238                 gyro_raw[2],
239                 time.time(),
240             )
241         )
242
243
244 class CsvLogger:
245     def __init__(self, path: Optional[str]):
246         self.file = None
247         self.writer = None
248
249         if path:
250             self.file = open(path, "w", newline="", encoding="utf-8")
251             self.writer = csv.writer(self.file)
252             self.writer.writerow(
253                 [
254                     "time",
255                     "ax",
256                     "ay",
257                     "az",
258                     "gx",
259                     "gy",
260                     "gz",
261                     "px",
262                     "py",
263                     "pz",
264                     "roll",
265                     "pitch",
266                     "yaw",
267                     "calibrated",
268                 ]
269             )
270
271     def write(self, s: ImuSample, p: np.ndarray, R: np.ndarray, calibrated: bool) -> None:
272         if self.writer is None:
273             return
274
275         roll, pitch, yaw = rot_to_rpy(R)
276
277         self.writer.writerow(
278             [
279                 s.t,
280                 s.ax,
281                 s.ay,
282                 s.az,
283                 s.gx,
284                 s.gy,
285                 s.gz,
286                 p[0],
287                 p[1],
288                 p[2],
289                 roll,
290                 pitch,
```

```
                yaw,
                int(calibrated),
            ]
        )

    def close(self) -> None:
        if self.file:
            self.file.close()


class ImuEstimator:
    def __init__(self):
        self.R = np.eye(3)
        self.v = np.zeros(3)
        self.p = np.zeros(3)

        self.acc_bias = np.zeros(3)
        self.gyro_bias = np.zeros(3)

        self.calibrated = False

        self.acc_calib = []
        self.gyro_calib = []

        self.last_t: Optional[float] = None

        self.still_buffer = deque()
        self.last_reset_t = 0.0

    def raw_to_units(self, s: ImuSample) -> Tuple[np.ndarray, np.ndarray]:
        acc = np.array([s.ax, s.ay, s.az], dtype=float) * ACC_SCALE
        gyro = np.array([s.gx, s.gy, s.gz], dtype=float) * GYRO_SCALE
        return acc, gyro

    def calibrate(self, acc: np.ndarray, gyro: np.ndarray) -> bool:
        if self.calibrated:
            return True

        self.acc_calib.append(acc)
        self.gyro_calib.append(gyro)

        if len(self.acc_calib) < CALIBRATION_SAMPLES:
            return False

        acc_mean = np.mean(np.asarray(self.acc_calib), axis=0)
        gyro_mean = np.mean(np.asarray(self.gyro_calib), axis=0)

        self.R = rot_between(acc_mean, -G)
        self.gyro_bias = gyro_mean
        self.acc_bias = acc_mean - self.R.T @ (-G)

        self.v[:] = 0.0
        self.p[:] = 0.0

        self.calibrated = True

        return True

    def update_still_buffer(self, t: float, acc: np.ndarray, gyro: np.ndarray) -> None:
        self.still_buffer.append((t, acc.copy(), gyro.copy()))

        while self.still_buffer and t - self.still_buffer[0][0] > STILL_WINDOW_SEC:
            self.still_buffer.popleft()

    def auto_reset_if_still(self, t: float) -> None:
        if len(self.still_buffer) < 25:
            return

        if self.still_buffer[-1][0] - self.still_buffer[0][0] < 0.8 * STILL_WINDOW_SEC:
            return

        accs = np.asarray([x[1] for x in self.still_buffer])
        gyros = np.asarray([x[2] for x in self.still_buffer])
```

```
364
365         gyro_level = float(np.max(np.linalg.norm(gyros - self.gyro_bias, axis=1)))
366         acc_range = float(np.max(np.max(accs, axis=0) - np.min(accs, axis=0)))
367
368         if (
369             gyro_level < math.radians(6.0)
370             and acc_range < 0.45
371             and t - self.last_reset_t > RESET_COOLDOWN_SEC
372         ):
373             acc_mean = np.mean(accs, axis=0)
374             gyro_mean = np.mean(gyros, axis=0)
375
376             self.R = rot_between(acc_mean, -G)
377             self.gyro_bias = gyro_mean
378             self.acc_bias = acc_mean - self.R.T @ (-G)
379
380             self.v[:] = 0.0
381             self.p[:] = 0.0
382
383             self.last_reset_t = t
384
385     def step(self, sample: ImuSample) -> Tuple[np.ndarray, np.ndarray, bool]:
386         acc_m, gyro_m = self.raw_to_units(sample)
387
388         self.update_still_buffer(sample.t, acc_m, gyro_m)
389
390         if not self.calibrate(acc_m, gyro_m):
391             return self.p, self.R, False
392
393         if self.last_t is None:
394             self.last_t = sample.t
395             return self.p, self.R, True
396
397         dt = min(max(sample.t - self.last_t, MIN_DT), MAX_DT)
398         self.last_t = sample.t
399
400         acc = acc_m - self.acc_bias
401         gyro = gyro_m - self.gyro_bias
402
403         self.R = self.R @ exp_so3(gyro * dt)
404
405         if abs(np.linalg.norm(acc) - 9.80665) < 0.9 and np.linalg.norm(gyro) < math.radians(30.0):
406             measured_up = normalize(self.R @ normalize(acc))
407             true_up = normalize(-G)
408             error_axis = np.cross(measured_up, true_up)
409             correction = min(GRAVITY_CORRECTION_GAIN * dt, 0.06) * error_axis
410             self.R = exp_so3(correction) @ self.R
411
412         a_world = self.R @ acc + G
413
414         if np.linalg.norm(a_world) < ACC_DEADBAND and np.linalg.norm(gyro) < math.radians(5.0):
415             a_world[:] = 0.0
416             self.v[:] = 0.0
417             self.p[:] = 0.0
418         elif np.linalg.norm(a_world) < ACC_DEADBAND:
419             a_world[:] = 0.0
420         self.p = self.p + self.v * dt + 0.5 * a_world * dt * dt
421         self.v = VELOCITY_DAMPING * (self.v + a_world * dt)
422
423         self.auto_reset_if_still(sample.t)
424
425         return self.p, self.R, True
426
427
428 class Viewer(QOpenGLWidget):
429     def __init__(self):
430         super().__init__()
431
432         self.setMinimumSize(1000, 700)
433
434         self.p = np.zeros(3)
435         self.R = np.eye(3)
436         self.calibrated = False
```

```
437
438         self.path = deque(maxlen=600)
439
440     def set_state(self, p: np.ndarray, R: np.ndarray, calibrated: bool) -> None:
441         self.p = p * POSITION_DISPLAY_GAIN
442         self.R = R.copy()
443         self.calibrated = calibrated
444
445         if calibrated:
446             self.path.append(self.p.copy())
447
448         self.update()
449
450     def initializeGL(self) -> None:
451         glClearColor(0.08, 0.09, 0.11, 1.0)
452         glEnable(GL_DEPTH_TEST)
453
454     def resizeGL(self, w: int, h: int) -> None:
455         glViewport(0, 0, w, max(1, h))
456         glMatrixMode(GL_PROJECTION)
457         glLoadIdentity()
458         gluPerspective(45.0, w / max(1, h), 0.01, 100.0)
459         glMatrixMode(GL_MODELVIEW)
460
461     def paintGL(self) -> None:
462         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
463         glLoadIdentity()
464
465         gluLookAt(3.2, -5.0, 3.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0)
466
467         self.draw_grid()
468         self.draw_axes()
469         self.draw_path()
470         self.draw_platform()
471
472     def draw_grid(self) -> None:
473         glColor3f(0.35, 0.35, 0.35)
474
475         glBegin(GL_LINES)
476
477         for i in range(-10, 11):
478             x = i * 0.2
479
480             glVertex3f(x, -2.0, 0.0)
481             glVertex3f(x, 2.0, 0.0)
482
483             glVertex3f(-2.0, x, 0.0)
484             glVertex3f(2.0, x, 0.0)
485
486         glEnd()
487
488     def draw_axes(self) -> None:
489         glLineWidth(2.5)
490
491         glBegin(GL_LINES)
492
493         glColor3f(1.0, 0.2, 0.2)
494         glVertex3f(0, 0, 0)
495         glVertex3f(0.8, 0, 0)
496
497         glColor3f(0.2, 1.0, 0.2)
498         glVertex3f(0, 0, 0)
499         glVertex3f(0, 0.8, 0)
500
501         glColor3f(0.2, 0.4, 1.0)
502         glVertex3f(0, 0, 0)
503         glVertex3f(0, 0, 0.8)
504
505         glEnd()
506
507         glLineWidth(1.0)
508
509     def draw_path(self) -> None:
```

```
        if len(self.path) < 2:
            return

        glColor3f(1.0, 0.8, 0.1)

        glBegin(GL_LINE_STRIP)

        for p in self.path:
            glVertex3f(float(p[0]), float(p[1]), float(p[2]))

        glEnd()

    def draw_box(self, lx: float, ly: float, lz: float) -> None:
        x = lx / 2.0
        y = ly / 2.0
        z = lz / 2.0

        glBegin(GL_QUADS)

        glVertex3f(-x, -y, z)
        glVertex3f(x, -y, z)
        glVertex3f(x, y, z)
        glVertex3f(-x, y, z)

        glVertex3f(-x, -y, -z)
        glVertex3f(-x, y, -z)
        glVertex3f(x, y, -z)
        glVertex3f(x, -y, -z)

        glVertex3f(-x, y, -z)
        glVertex3f(-x, y, z)
        glVertex3f(x, y, z)
        glVertex3f(x, y, -z)

        glVertex3f(-x, -y, -z)
        glVertex3f(x, -y, -z)
        glVertex3f(x, -y, z)
        glVertex3f(-x, -y, z)

        glVertex3f(x, -y, -z)
        glVertex3f(x, y, -z)
        glVertex3f(x, y, z)
        glVertex3f(x, -y, z)

        glVertex3f(-x, -y, -z)
        glVertex3f(-x, -y, z)
        glVertex3f(-x, y, z)
        glVertex3f(-x, y, -z)

        glEnd()

    def draw_rotor(self, x: float, y: float) -> None:
        glPushMatrix()
        glTranslatef(x, y, 0.02)

        glBegin(GL_TRIANGLE_FAN)
        glVertex3f(0.0, 0.0, 0.0)

        for k in range(33):
            a = 2.0 * math.pi * k / 32.0
            glVertex3f(0.12 * math.cos(a), 0.12 * math.sin(a), 0.0)

        glEnd()

        glColor3f(0.12, 0.12, 0.12)

        glBegin(GL_LINES)

        glVertex3f(-0.18, 0.0, 0.01)
        glVertex3f(0.18, 0.0, 0.01)

        glVertex3f(0.0, -0.18, 0.01)
        glVertex3f(0.0, 0.18, 0.01)
```

```
583
584         glEnd()
585
586         glPopMatrix()
587
588     def draw_platform(self) -> None:
589         glPushMatrix()
590
591         glTranslatef(float(self.p[0]), float(self.p[1]), float(self.p[2]))
592
593         roll, pitch, yaw = rot_to_rpy(self.R)
594         display_R = rpy_to_rot(-roll, -pitch, yaw)
595
596         M = np.eye(4, dtype=np.float32)
597         M[:3, :3] = display_R
598
599         glMultMatrixf(M.T)
600
601         glColor3f(0.18, 0.68, 0.75)
602         self.draw_box(0.42, 0.24, 0.12)
603
604         glLineWidth(5.0)
605         glColor3f(0.78, 0.78, 0.78)
606
607         glBegin(GL_LINES)
608
609         glVertex3f(-0.55, 0.0, 0.0)
610         glVertex3f(0.55, 0.0, 0.0)
611
612         glVertex3f(0.0, -0.55, 0.0)
613         glVertex3f(0.0, 0.55, 0.0)
614
615         glEnd()
616
617         glLineWidth(1.0)
618
619         glColor3f(0.96, 0.56, 0.20)
620         self.draw_rotor(0.55, 0.0)
621
622         glColor3f(0.44, 0.76, 0.28)
623         self.draw_rotor(-0.55, 0.0)
624
625         glColor3f(0.70, 0.35, 0.85)
626         self.draw_rotor(0.0, 0.55)
627
628         glColor3f(0.95, 0.80, 0.25)
629         self.draw_rotor(0.0, -0.55)
630
631         glColor3f(1.0, 1.0, 1.0)
632
633         glBegin(GL_TRIANGLES)
634
635         glVertex3f(0.32, 0.0, 0.09)
636         glVertex3f(0.12, 0.08, 0.09)
637         glVertex3f(0.12, -0.08, 0.09)
638
639         glEnd()
640
641         glPopMatrix()
642
643
644 class MainWindow(QtWidgets.QMainWindow):
645     def __init__(self, args):
646         super().__init__()
647
648         self.setWindowTitle("Real-Time IMU Motion Viewer")
649
650         self.viewer = Viewer()
651         self.setCentralWidget(self.viewer)
652
653         self.estimator = ImuEstimator()
654
655         self.samples: queue.Queue[ImuSample] = queue.Queue(maxsize=3000)
```

```
656
657         self.reader: Optional[SerialWorker] = None
658         self.demo: Optional[DemoSource] = None
659
660         self.logger = CsvLogger(args.log)
661
662         if args.demo:
663             self.demo = DemoSource()
664             self.demo.new_sample.connect(self.handle_sample)
665
666         else:
667             if not args.port:
668                 raise ValueError("Please provide --port, for example --port COM3")
669
670             self.reader = SerialWorker(args.port, args.baud, self.samples)
671             self.reader.start()
672
673         self.timer = QtCore.QTimer()
674         self.timer.timeout.connect(self.consume_samples)
675         self.timer.start(16)
676
677         self.statusBar().showMessage("Keep the IMU still for calibration...")
678
679     def consume_samples(self) -> None:
680         for _ in range(250):
681             try:
682                 sample = self.samples.get_nowait()
683             except queue.Empty:
684                 break
685
686             self.handle_sample(sample)
687
688     def handle_sample(self, sample: ImuSample) -> None:
689         p, R, calibrated = self.estimator.step(sample)
690
691         self.viewer.set_state(p, R, calibrated)
692         self.logger.write(sample, p, R, calibrated)
693
694         if calibrated:
695             roll, pitch, yaw = rot_to_rpy(R)
696
697             self.statusBar().showMessage(
698                 f"calibrated | p=[{p[0]:+.3f}, {p[1]:+.3f}, {p[2]:+.3f}] m | "
699                 f"rpy=[{math.degrees(roll):+.1f}, {math.degrees(pitch):+.1f}, {math.degrees(yaw):+.1f}]
      deg"
700             )
701
702         else:
703             n = len(self.estimator.acc_calib)
704             self.statusBar().showMessage(f"calibrating {n}/{CALIBRATION_SAMPLES}: keep the MPU6050 still")
705
706     def keyPressEvent(self, event) -> None:
707         if event.key() == QtCore.Qt.Key.Key_S:
708             name = time.strftime("imu_snapshot_%Y%m%d_%H%M%S.png")
709             self.viewer.grabFramebuffer().save(name)
710             self.statusBar().showMessage(f"saved {name}")
711
712         else:
713             super().keyPressEvent(event)
714
715     def closeEvent(self, event) -> None:
716         if self.reader is not None:
717             self.reader.stop()
718
719         self.logger.close()
720
721         super().closeEvent(event)
722
723
724 def parse_args():
725     parser = argparse.ArgumentParser(description="Real-time IMU motion visualization")
726
727     parser.add_argument("--port", type=str, default=None)
```

```
728     parser.add_argument("--baud", type=int, default=500000)
729     parser.add_argument("--demo", action="store_true")
730     parser.add_argument("--log", type=str, default=None)
731 
732     return parser.parse_args()
733 
734 
735 def main() -> None:
736     args = parse_args()
737 
738     app = QtWidgets.QApplication(sys.argv)
739 
740     win = MainWindow(args)
741     win.show()
742 
743     sys.exit(app.exec())
744 
745 
746 if __name__ == "__main__":
747     main()
```

Listing 2: Complete Python source code: MPU_6050_visualization.py.